SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO5 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 3 – Short Answer Test
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	Shahana	**Reg. No.:**	192524479
Section / Batch:	Slot A	**Date:**	04/09/2026

Code of Conduct

I Shahana (192524479) *(Name / Reg No)*

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shahana

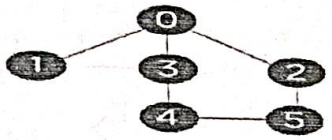
Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Short Answer Test

1. Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop 0, 3) Pop 0, 3) Pop Construct AVL tree for the following elements. Mention the
2. Given a collection of elements 10,20,30,40,50,60,70,80. The Key element to be search 70. Compare how searching is performed in Linear Search and Binary Search. Identify the efficient search method. Justify.
3. What is the postfix and prefix forms of the following expression? A+B*(C-D)/(P-R)
4. Construct AVL tree for the following elements. Mention the Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10
5. Construct a heap by inserting 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45 into a binary heap.
6. Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
7. Perform the merge sort for the following inputs. Also, show the result for each step of iteration. 40,20,70,14,60,61,97,30.
8. How the following data set can be sorted by quick sort. Show each step-in sorting. 78,23,31,88,43,55,67,55.
9. Perform Open Hashing & Linear Probing for the given set of 0,1,81, 4,64,25,16,36, 9 and 49.
10. Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the allotted time	Completes most questions on time	Requires additional time	Unable to complete the test

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Stack Operations

Stack: 22, 55, 33, 66, 88 top: 88.

i, Invert the elements — 88, 66, 33, 55, 22

ii, Pop() — Removes top element (22) → 88, 66, 33, 55

iii, Pop() — Removes top element (55) → 88, 66, 33

iv, Pop() — Removes top element (33) → 88, 66

Final stack: 88, 66

2. Linear Search & Binary Search.

Array: 10, 20, 30, 40, 50, 60, 70, 80 (n=8), Search key = 70

Linear Search: Check sequentially through 10 → 20 → 30 → 40 → 50 → 60 → 70 (found) within 7 comparisons.

Time Complexity: O(n)

Binary Search: (array must be sorted).

- low = 0, high = 7, mid = 3 → arr[3] = 40 < 70 → Search right
- low = 4, high = 7, mid = 5 → arr[5] = 60 < 70 → Search right
- low = 6, high = 7, mid = 6 → arr[6] = 70 = 70 → found

3 comparisons. Time complexity O(log n)

Efficient method:

Binary Search - It took 3 comparisons versus 7, since it eliminates half the search space each step (O(log n)) vs (O(n)). Justified for sorted, static datasets.

3. Prefix and Postfix of A+B * (C - D) / (P - R)

Step 1 - Infix: A+B * (C - D) / (P - R)

Operator Precedence: Inside brackets first, then * and / (left to right, same precedence), then +

Postfix:

AB CD - + PR - /+

Prefix (Polish):

+ A/* B - CD - PR

4. AVL Tree construction: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

~~Increasi~~ Inserting sequential increasing values causes repeated Right-Right (RR) imbalance, fixed by Left Rotation.

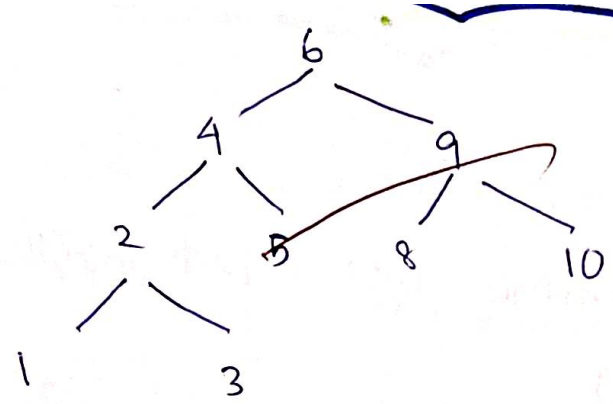
- Insert 1 : 1
- Insert 2: 1 → 2 (balanced)
- Insert 3: Unbalanced at 1 (RR case) → Left Rotate at 1 → Tree: 2(1,3)
- Insert 4: 2(1, 3, (-4)) — balanced Tree: 2(1, 3—,4)
- Insert 5: Unbalanced at 3 (RR) → Left Rotate on 3.

  Tree: 2(1, 4(3, 5))
- Insert 6 → Unbalanced at 2 (RR case, since right subtree height grows) → Left Rotate on 2 → Tree: 4(2(1,3), 5(-;6))
- Insert 7 → Unbalanced at 5 (RR), Left Rotate on 5 →

  Tree: 4(2(1,3), 6(5,7))
- Insert 8 → Unbalanced at 6 (RR). Left rotate on 6 → Tree: 4(2(1,3), 6(5,7)) → then unbalanced at 4 (RR at root level) → Left rotate on 4 → Tree: 6(4(2(1,3),5),7(-,8))
- Insert 9 → Unbalanced at 7 (RR) → Left Rotate on 7 →

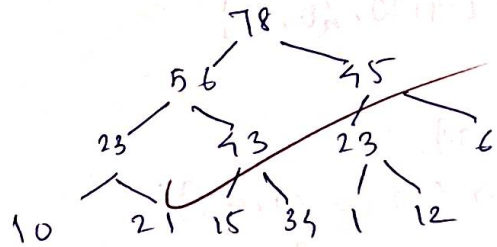
  Tree: 6(4(2(1,3),5), 8(7,9))
- Insert 10 → Unbalanced at 8 (RR) → Left Rotate on 8 →

  Tree: 6(4(2(1,3), 5), 9(8,10))



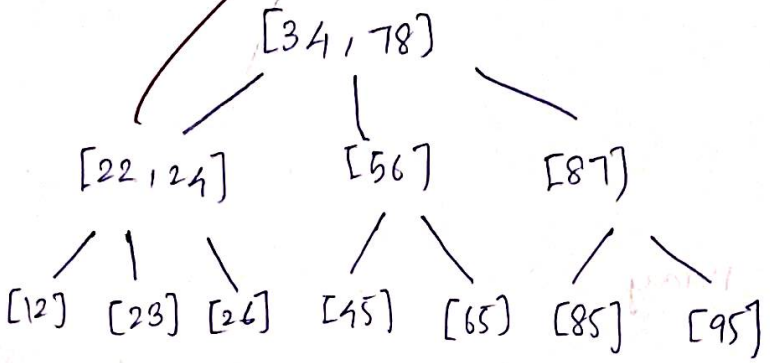
5. Binary Heap

Insert	Heap Array
10	[10]
15	[15, 10]
12	[15, 10, 12]
23	[23, 15, 12, 10]
21	[23, 21, 12, 10, 15, 1]
6	[23, 21, 12, 10, 15, 1, 6]
43	[43, 23, 12, 21, 15, 1, 6, 10]
34	[43, 34, 12, 23, 15, 1, 6, 10, 21]
56	[56, 43, 12, 23, 34, 1, 6, 10, 21, 15]
78	[78, 56, 12, 23, 43, 1, 6, 10, 21, 15, 34]
23	[78, 56, 23, 23, 43, 12, 6, 10, 21, 15, 34, 1]
45	[78, 56, 45, 23, 43, 23, 6, 10, 21, 15, 34, 1, 12]



6. B Tree of Order 3

- Max 2 keys per node
- If there is an overflow: split → push middle key up.



7. Merge sort

Divide:

- [40, 20, 70, 14, 60, 61, 97, 30] → [40, 20, 70, 14] & [60, 61, 97, 30]
- [40, 20, 70, 14] → [40, 20] and [70, 14]
- [40, 20] → [40] and [20]
- [70, 14] → [70] and [14]
- [60, 61, 97, 30] → [60, 61] and [97, 30]
- [60, 61] → [60] and [61]
- [97, 30] → [97] and [30]

Merge:

- [40] + [20] → [20, 40]
- [70] + [14] → [14, 70]
- [20, 40] + [14, 70] → [14, 20, 40, 70]
- [60] + [61] → [60, 61]
- [97] + [30] → [30, 97]
- [60, 61] + [30, 97] → [30, 60, 61, 97]
- [14, 20, 40, 70] + [30, 60, 61, 97] → [14, 20, 30, 40, 60, 61, 70, 97]

Quick Sort

- [78, 23, 31, 88, 43, 55, 67, 55], pivot = 55 → [23, 31, 43, 55, 55, 88, 67, 78]
- Left [23, 31, 43, 55] pivot = 55 → already sorted position.
  - [23, 31, 43] pivot = 43 → sorted
  - [23, 31] pivot = 31 → sorted.
    - Right [88, 67, 78] pivot = 78 → [67, 78, 88]

Final sorted array : 23, 31, 43, 55, 55, 67, 78, 88.

9. Hashing.

Table size = 10.

h(k) = k mod 10.

Open Hashing:

0 → 0 | 1 → 1, 81 | 2, 3 → empty | 4 → 4, 64 | 5 → 25 | 6 → 16, 36 | 7, 8 →

empty | 9 → 9, 49.

Linear Probing:

Key	Home	Placed
0	0	0
1	1	1
81	1	2
4	4	4
64	4	5
25	5	6
16	6	7
36	6	8
9	9	9
49	9	3

Final Table:

[0, 1, 81, 49, 4, 64, 25, 16, 36, 9]

10. BFS on the graph

Edges: 0-1, 0-3, 0-2, 3-4, 2-5, 4-5.

Starting from node 0:

- Visit 0 → enqueue neighbours: 1, 3, 2.
- Dequeue 1 → no new neighbours.
- Dequeue 3 → enqueue neighbour 4
- Dequeue 2 → enqueue neighbour 5 (if not already visited)
- Dequeue 4 → neighbour 5 already visited / enqueued
- Dequeue 5 → done.

BFS Traversal order: 0 → 1 → 3 → 2 → 4 → 5.